

CSE/ISyE 6740
Computational Data Analysis

Logistic Regression and Support Vector Machine

09/16/2026

Kai Wang, Assistant Professor in Computational Science and Engineering
kwang692@gatech.edu

Outline

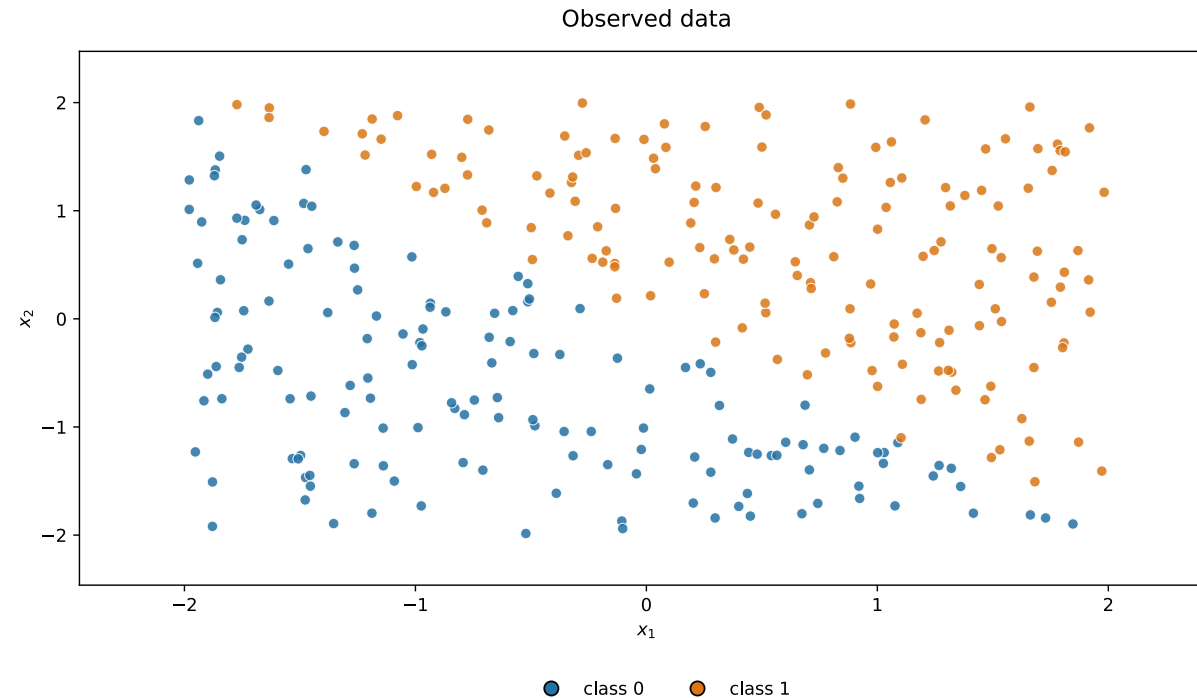
- **Supervised Learning**
 - **Decision boundary**
 - **K-nearest neighbors**
 - **Logistic regression**
 - **Support Vector Machine**

Decision Boundary

Decision-making: Divide High-dimensional Space

- Distributions of positive sample and negative samples
 - How to construct the classifier $f: X \rightarrow Y$?

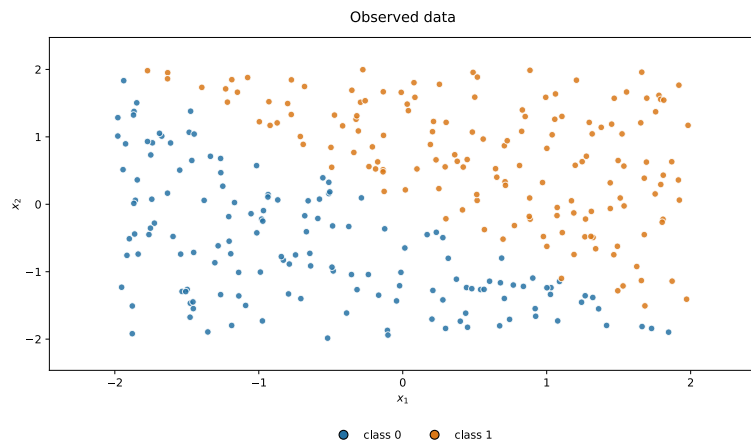
Synthetic: diagonal separation



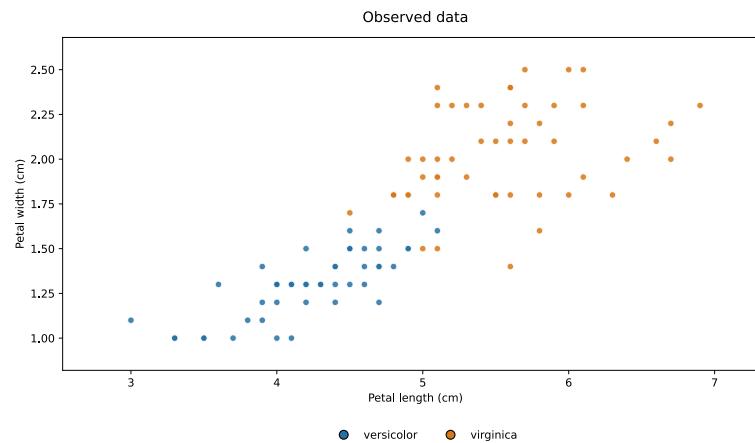
Decision Boundary

How should we separate the points?

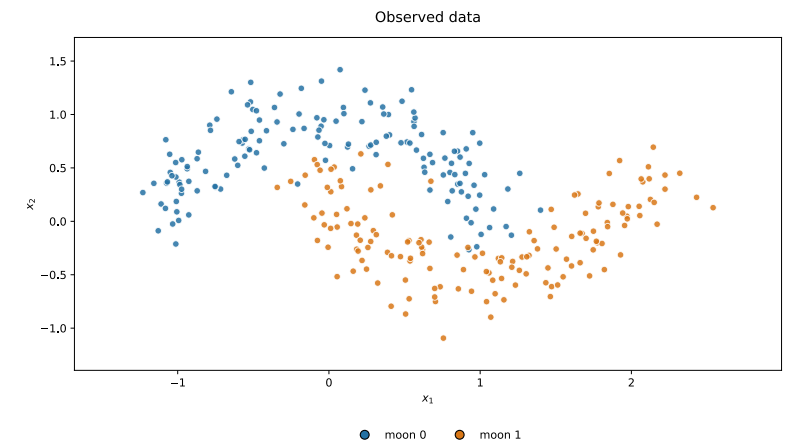
Synthetic: diagonal separation



Iris: two species, two measurements

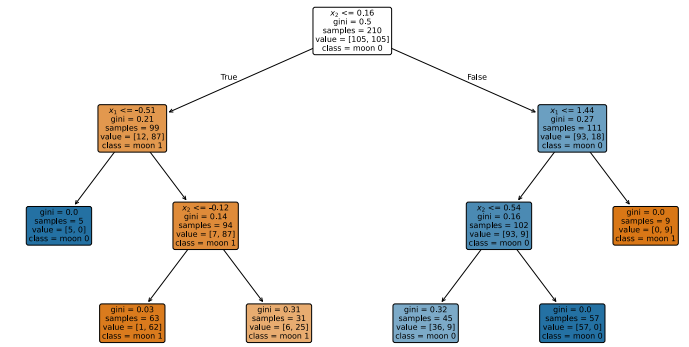


Synthetic: two interleaving moons



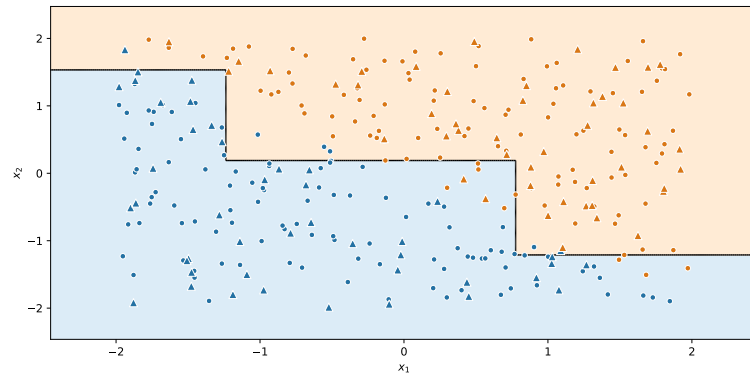
Decision Tree Boundary

Each node asks about ONE feature: Synthetic: two interleaving moons



Synthetic: diagonal separation

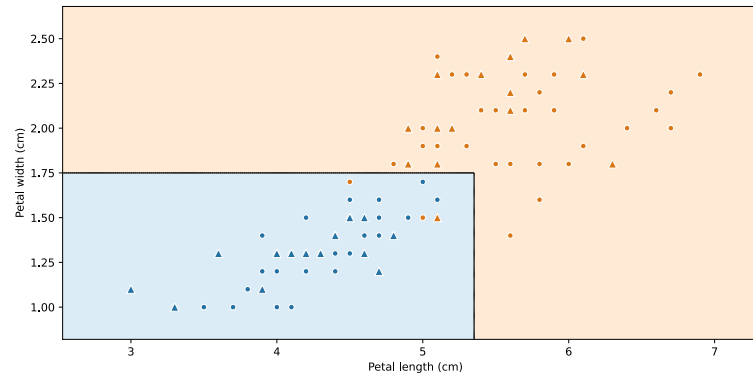
Decision tree (depth 3) | Train 94.8% | Test 95.6%



● class 0 ● class 1 ● Training ▲ Held-out test

Iris: two species, two measurements

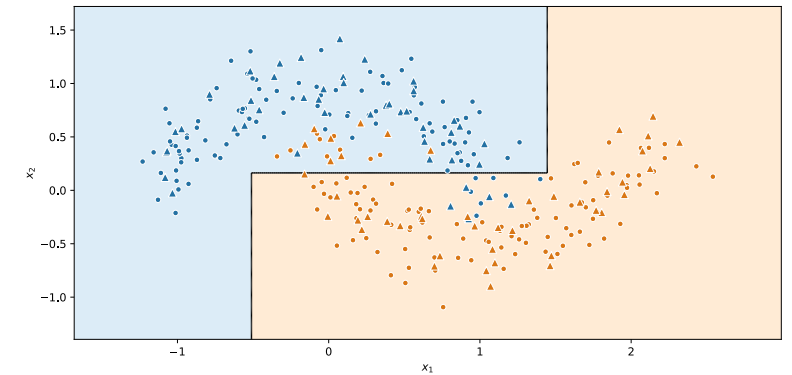
Decision tree (depth 3) | Train 95.7% | Test 96.7%



● versicolor ● virginica ● Training ▲ Held-out test

Synthetic: two interleaving moons

Decision tree (depth 3) | Train 92.4% | Test 85.6%



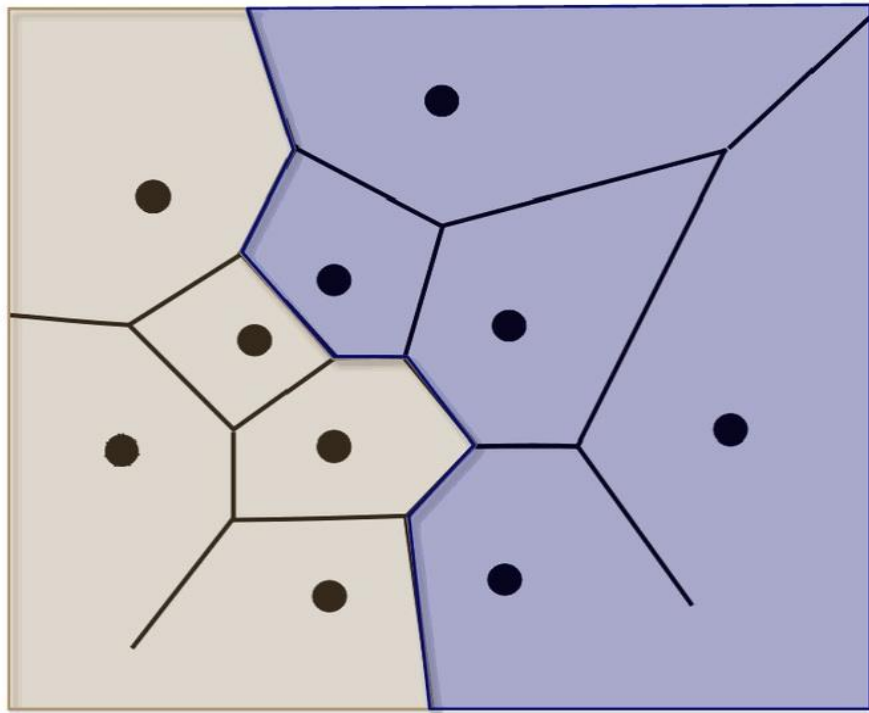
● moon 0 ● moon 1 ● Training ▲ Held-out test

Can we draw other kinds of decision boundary?

K-Nearest Neighbors

Nearest Neighbor Classifier

- The **nearest neighbor classifier**: assign x the same label as the closest training point x_i
- The nearest neighbor rule defines a **Voronoi partition** of the input space



- Parametric or non-parametric?
- Linear or nonlinear?
- Easy to kernelize

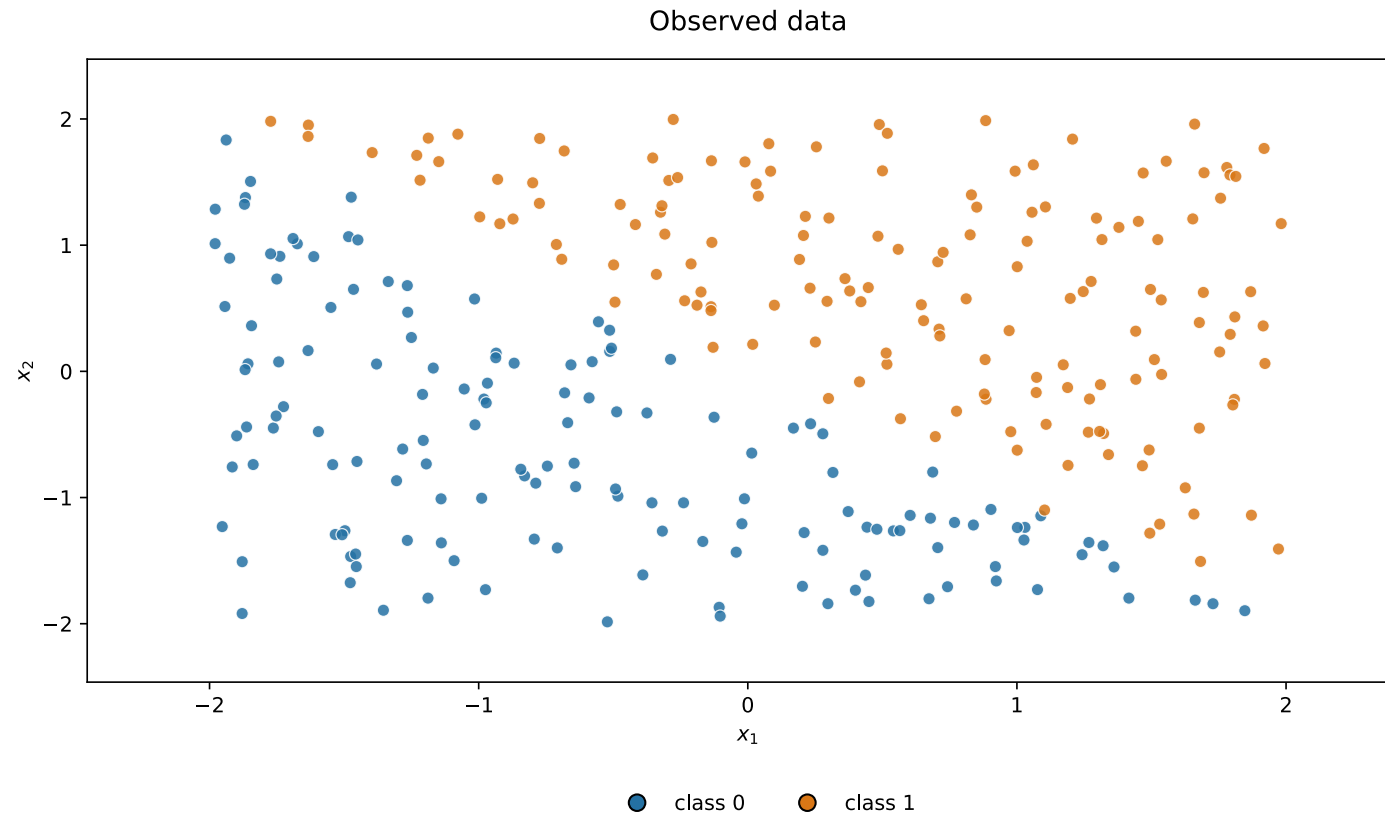
k -nearest Neighbor Classifier

- The **nearest neighbor classifier**: assign x a label by taking a majority vote over **the k training point x_i closest to x**
- For $k \geq 1$, the k -nearest neighbor rule generalizes the nearest neighbor rule
- To define this more mathematically:
 - $I_k(x) :=$ indices of the k nearest training points closest to x
 - If $y_i = \pm 1$, then we can write the k -nearest neighbor classifier as:

$$f_k(x) := \text{sign} \left(\sum_{i \in I_k(x)} y_i \right)$$

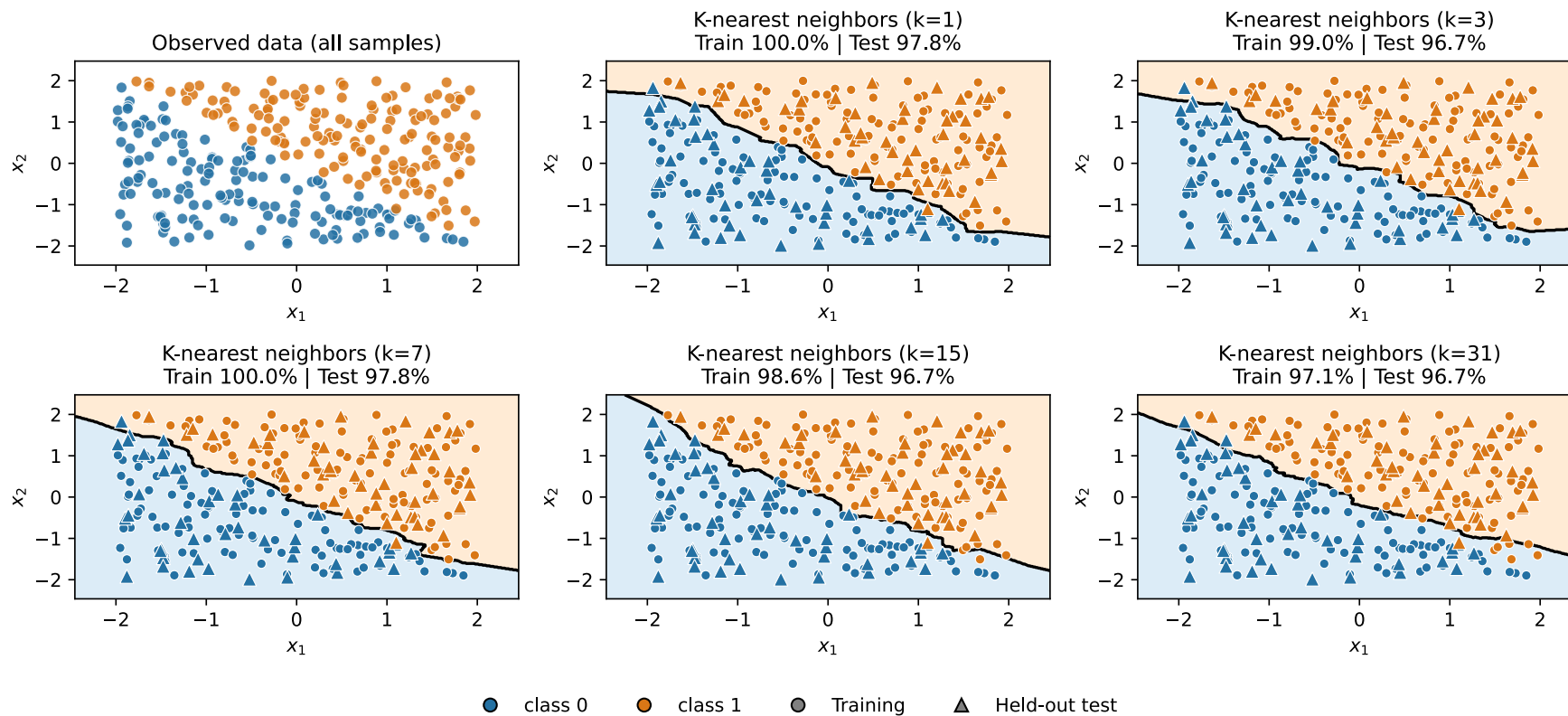
Example

Synthetic: diagonal separation



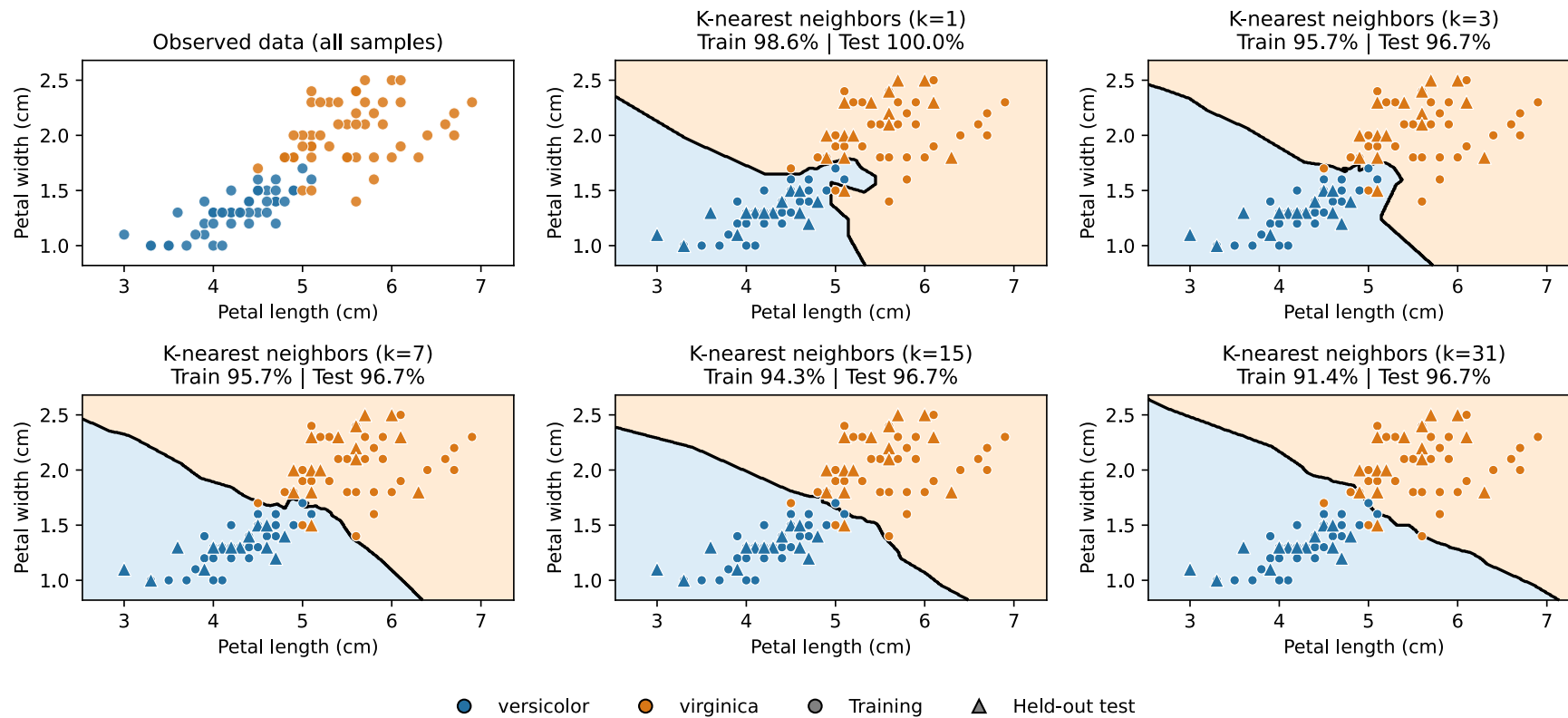
Example

Synthetic: diagonal separation
KNN: changing the number of neighbors



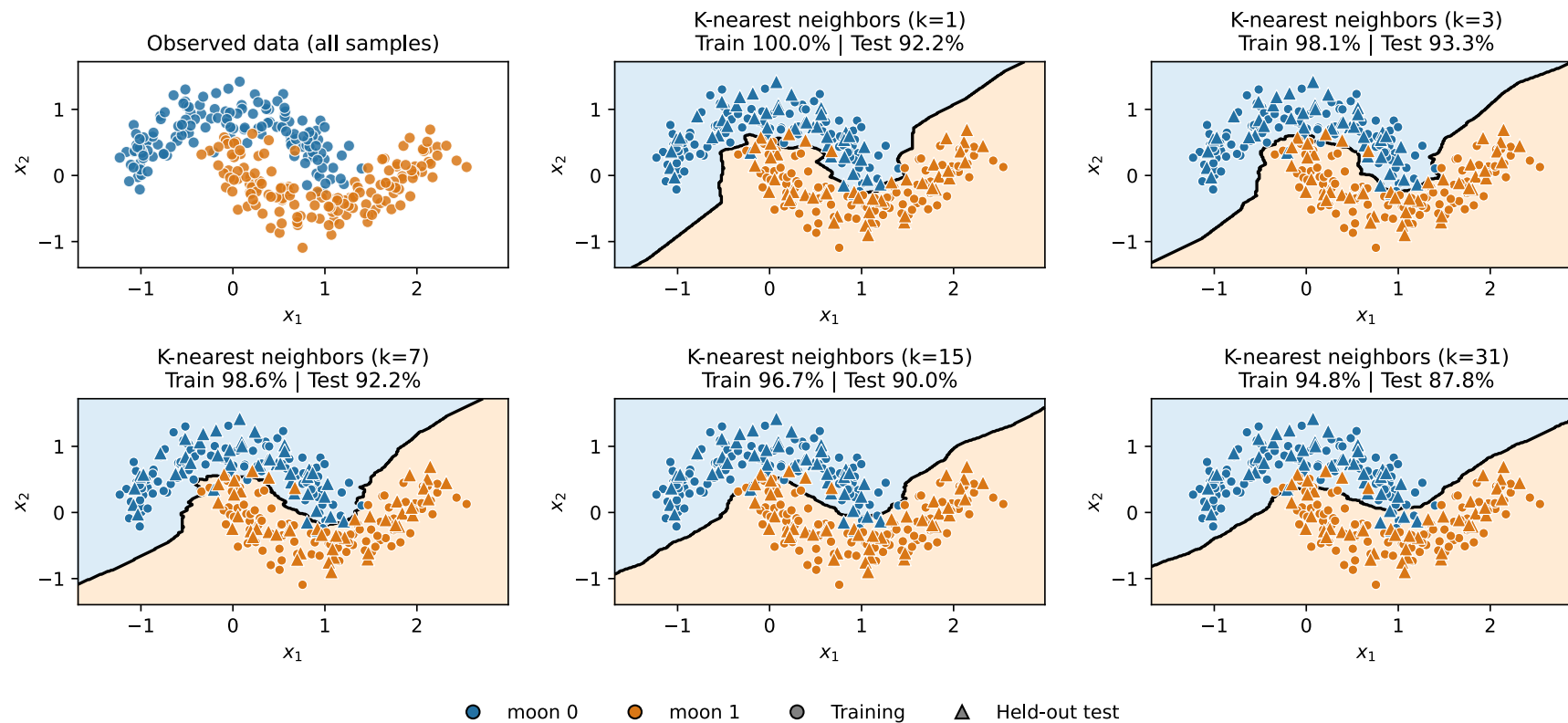
Example

Iris: two species, two measurements
KNN: changing the number of neighbors



Example

Synthetic: two interleaving moons
KNN: changing the number of neighbors



Choosing the Size of the Neighborhood

- Setting the parameter k is a problem of **model selection**
- Should we choose k with the smallest training ?

$$\hat{R}_n(f_k) = \frac{1}{n} \sum_{i=1}^n 1_{\{f_k(x_i) \neq y_i\}}$$

- No, by choosing $k = 1$, we always easily have training error $\hat{R}_n(f_1) = 0$
- Not much practical guidance from the theory, so we typically must rely on estimates based on holdout sets or more sophisticated model selection techniques.

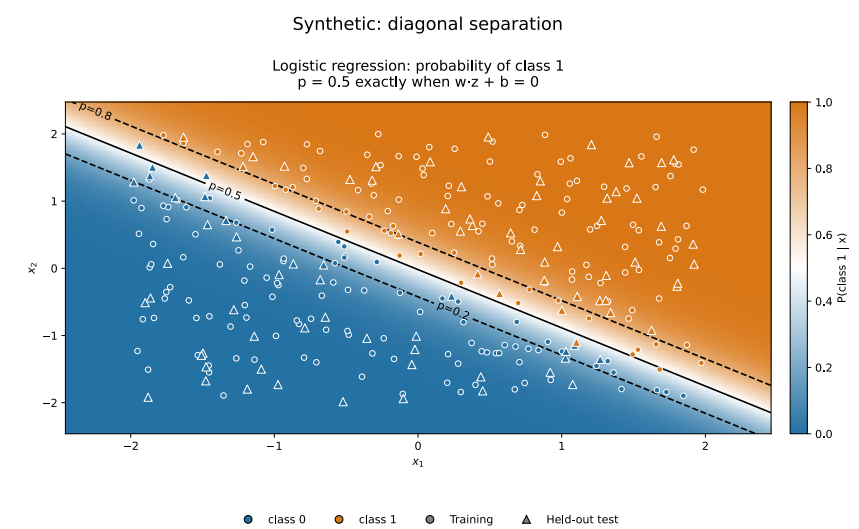
Computations in KNN

- Similar to KDE, essentially no “training” or “learning” phase. Computation is needed when applying the classifier
 - Memory: $O(nd)$ (why?)
- Finding the nearest neighbors out of a set of millions of examples is still pretty hard
 - Test computation: $O(nd)$ (why?)
- Use smart data structures and algorithms to index training data
 - Memory: $O(nd)$
 - Training computation (preprocessing): $O(n \log n)$
 - Test computation: $O(\log n)$
 - [K-D tree](#), [ball tree](#)

Logistic Regression

Discriminative Classifier

- Estimate decision boundary $h(x) = \log \frac{P(y = 1|x)}{P(y = 0|x)}$
 - $h(x) \geq 0 \Rightarrow$ positive, $h(x) < 0 \Rightarrow$ negative
 - Density estimation
 - Probabilistic meaning
- Model $h(x)$ or $f(x) := P(y = 1|x)$ as a function of x , and
 - Logistic regression, neural networks
 - Do **not** have probabilistic meaning
 - Hence can **not** be used to sample data points
- Why discriminative classifier?
 - Avoid difficult density estimation problem
 - Empirically achieve better classification results

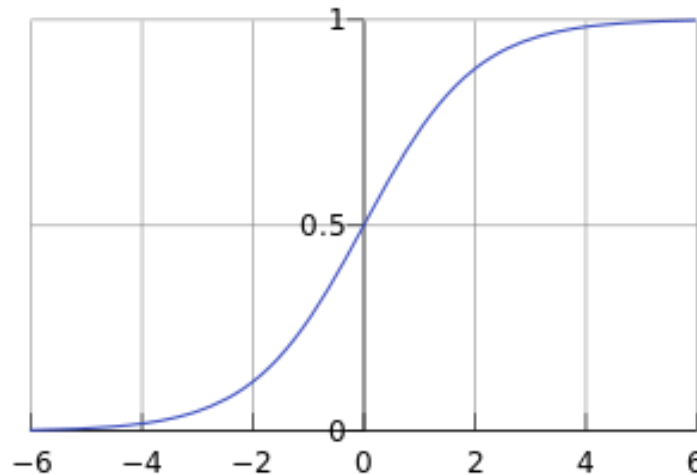


Logistic Regression Model

- Assume that the posterior distribution $P(y = 1|x)$ take a particular form:

$$P(y = 1|x, w) = \frac{1}{1 + \exp(-w^T x)}$$

- Logistic function $f(u) = \frac{1}{1 + \exp(-u)}$



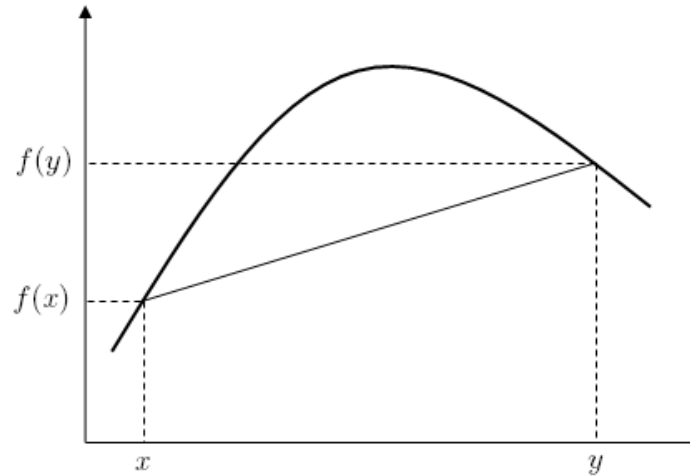
The larger $w^T x$, the higher the chance it is $y = 1$

Learning Parameters in Logistic Regression

- Find w , such that the conditional likelihood of the labels is maximized

$$\max_w l(w) := \log \prod_{i=1}^n P(y^i | x^i, w)$$

- Good news:** $l(w)$ is a concave function of w , and there is a single global optimum (if it exists).



- Bad news:** no closed form solution (we need to use numerical method)

The Objective Function $l(w)$

- Logistic regression model

$$p(y = 1|x, w) = \frac{1}{1 + \exp(-w^\top x)}$$

- Note that

$$p(y = 0|x, w) = 1 - \frac{1}{1 + \exp(-w^\top x)} = \frac{\exp(-w^\top x)}{1 + \exp(-w^\top x)}$$

- Plug in

$$\begin{aligned} l(w) &:= \log \prod_{i=1}^n p(y^i|x^i, w) \\ &= \sum_{i=1}^n [(1 - y^i) \log(1 - p) + y^i \log p] \end{aligned}$$

$$\begin{aligned} p &= p(y^i = 1|x^i, w) \\ 1 - p &= p(y^i = 0|x^i, w) \end{aligned}$$

Cross entropy loss for
2 classes

The Gradient of $l(w)$

$$\begin{aligned} l(w) &:= \log \prod_{i=1}^n p(y^i | x^i, w) \\ &= \sum_i (y^i - 1) w^\top x^i - \log(1 + \exp(-w^\top x^i)) \end{aligned}$$

- Gradient

$$\frac{\partial l(w)}{\partial w} = \sum_i (y^i - 1) x^i + \frac{\exp(-w^\top x^i) x^i}{1 + \exp(-w^\top x^i)}$$

- But setting it to 0 does not lead to closed form solution

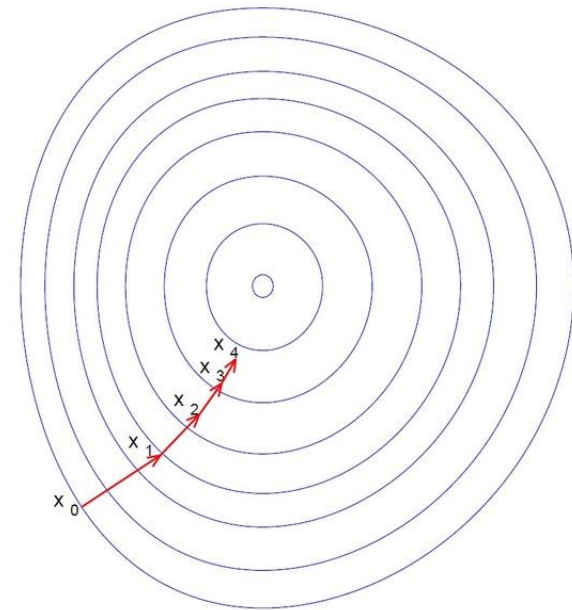
Gradient Descent

- Gradient descent is a popular and general method to solve optimization problems
- Given an initial guess, we iteratively refine the guess by taking the direction of the negative gradient
- Think about going down a hill by taking the steepest direction at each step

- Update rule

$$x_{k+1} = x_k - \gamma_k \nabla_x f(x_k)$$

γ_k is called the step size or learning rate



Gradient Ascent/Descent Algorithm

- Initialize parameter w^0
- Do

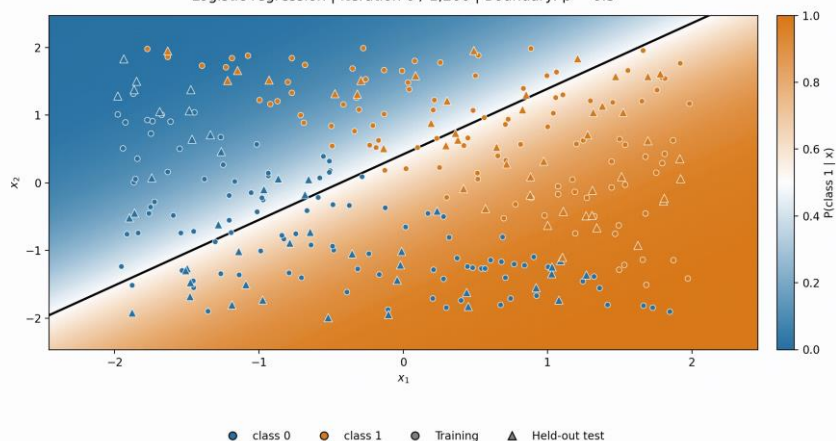
$$w^{t+1} \leftarrow w^t + \eta \sum_i (y^i - 1)x^i + \frac{\exp(-w^\top x^i)x^i}{1 + \exp(-w^\top x^i)}$$

- While $\|w^{t+1} - w^t\| > \epsilon$

Logistic Regression

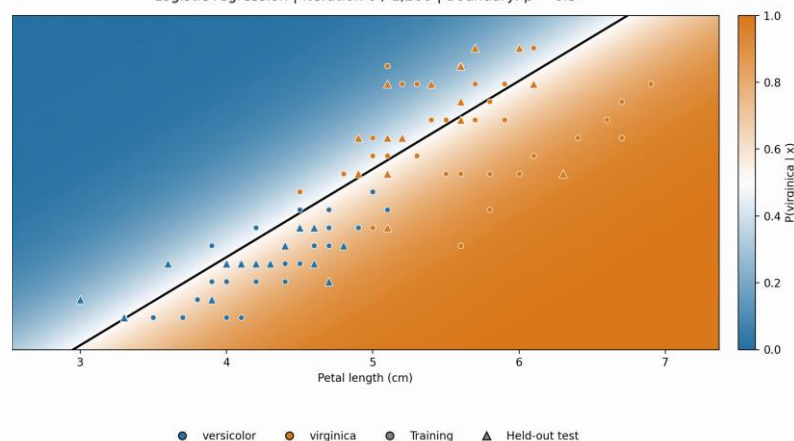
Synthetic: diagonal separation

Logistic regression | Iteration 0 / 1,200 | Boundary: $p = 0.5$



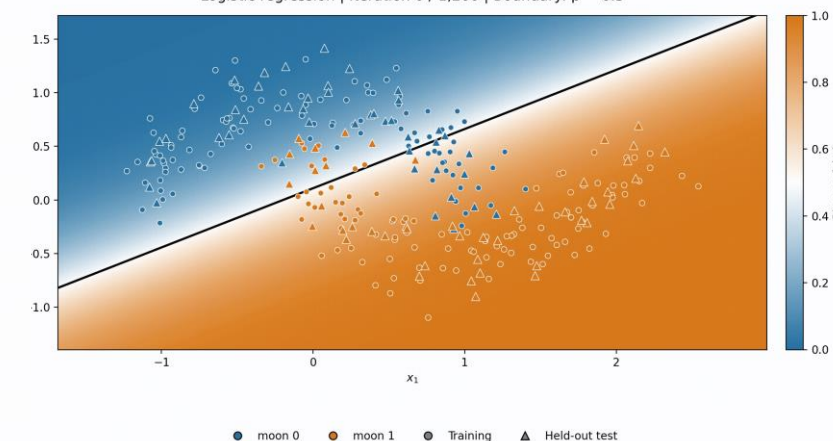
Iris: two species, two measurements

Logistic regression | Iteration 0 / 1,200 | Boundary: $p = 0.5$



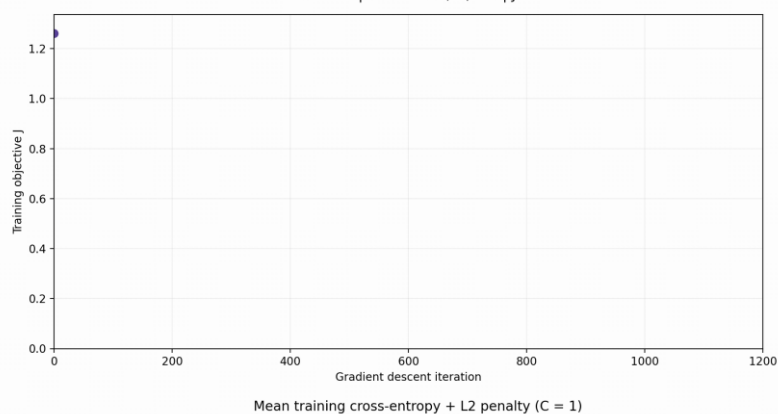
Synthetic: two interleaving moons

Logistic regression | Iteration 0 / 1,200 | Boundary: $p = 0.5$



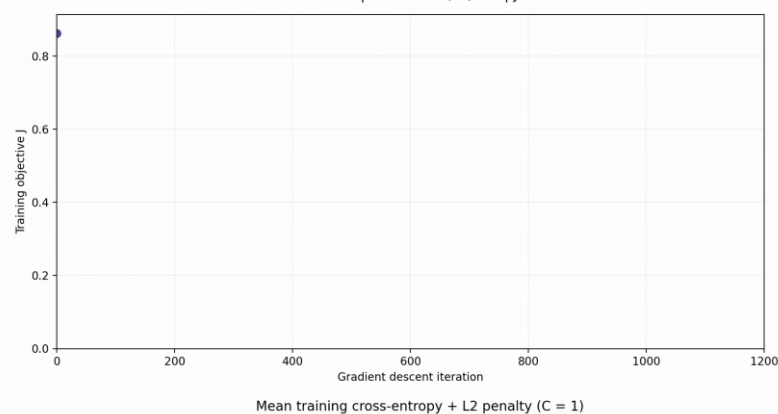
Synthetic: diagonal separation

Gradient descent | Iteration 0 / 1,200 | $J = 1.2610$



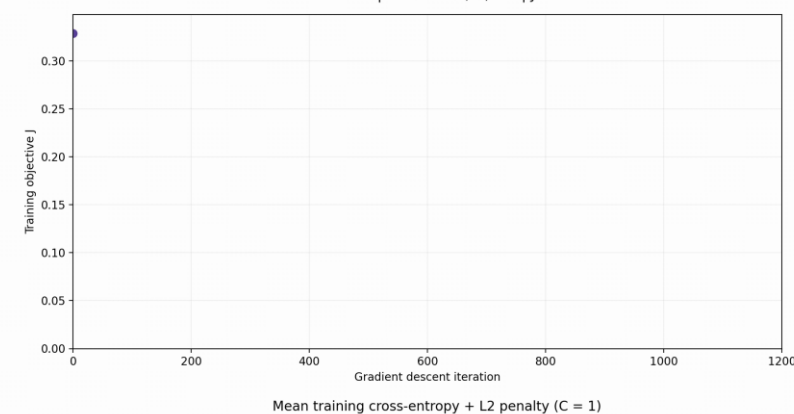
Iris: two species, two measurements

Gradient descent | Iteration 0 / 1,200 | $J = 0.8629$



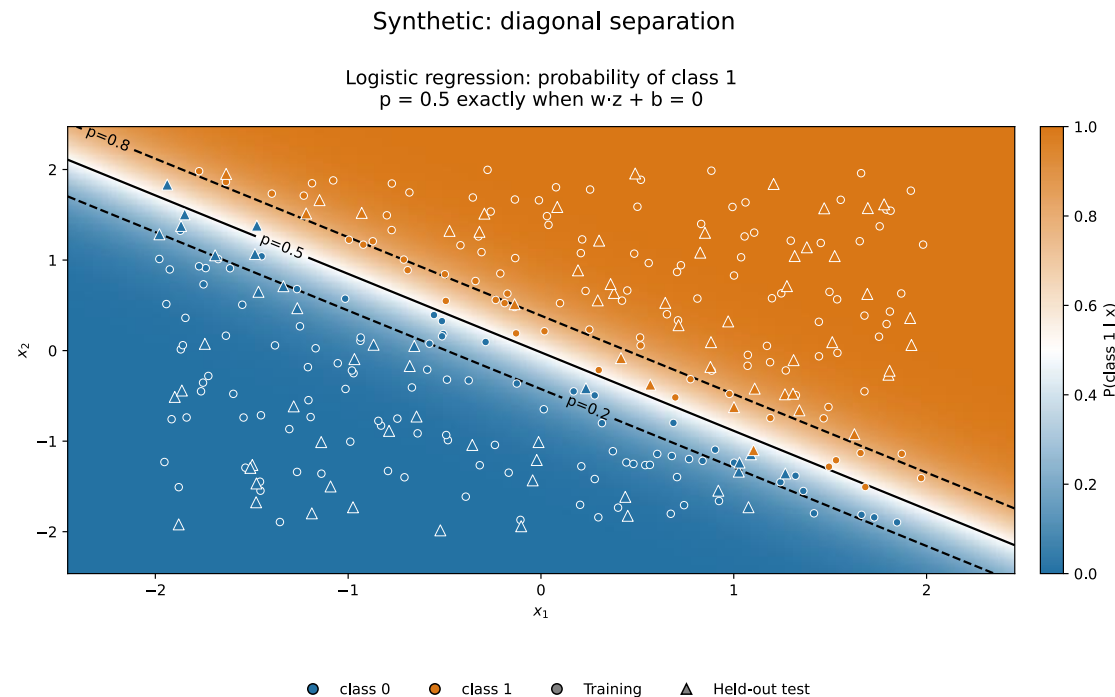
Synthetic: two interleaving moons

Gradient descent | Iteration 0 / 1,200 | $J = 0.3287$



Decision Boundary?

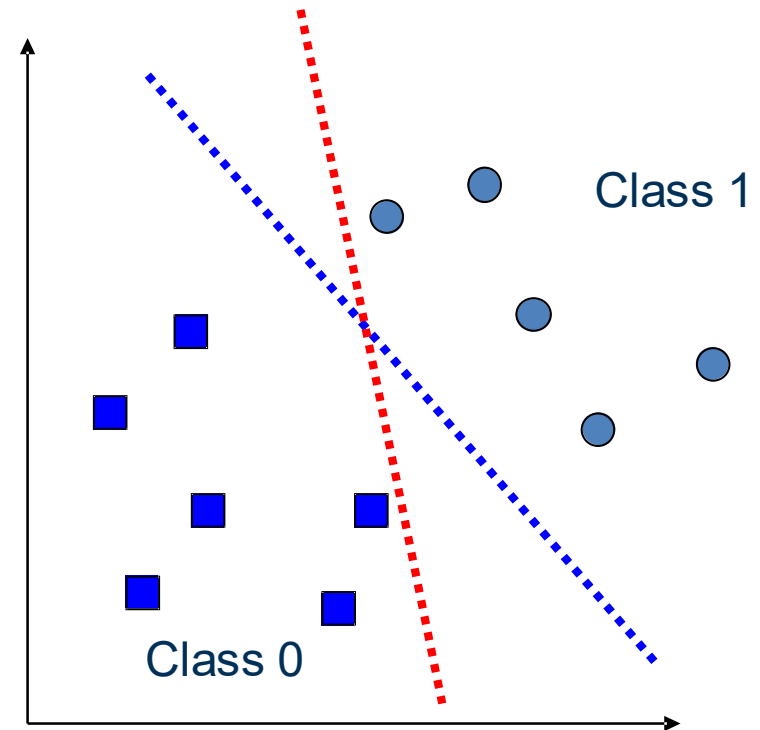
- What is the decision boundary of logistic regression?
- When is $h(x) = \log \frac{P(y = 1|x)}{P(y = 0|x)} = 0$?



Support Vector Machines

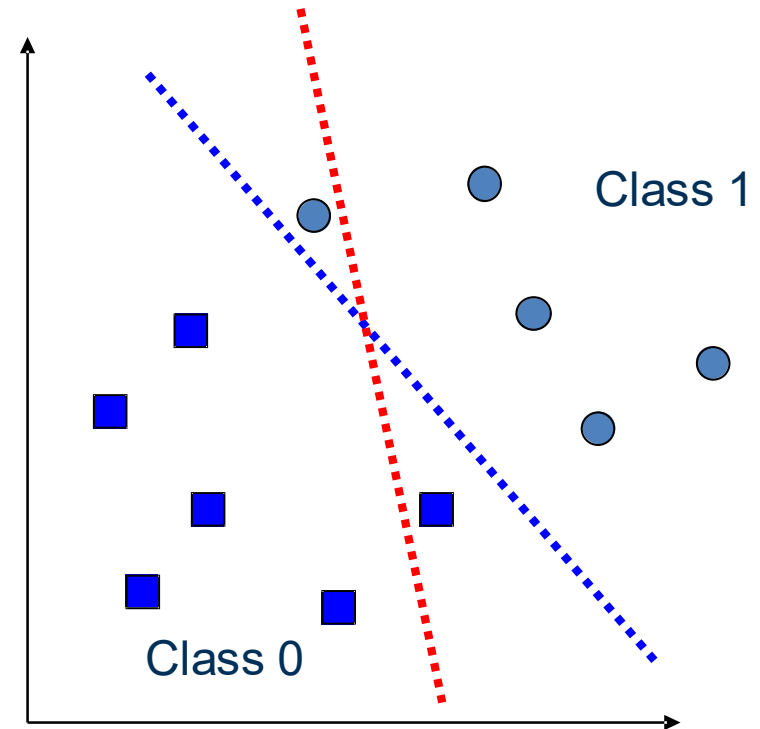
Which Decision Boundary is Better?

- Suppose the training samples are linearly separable
- We can find a decision boundary which gives zero **training** error
- But there are many such decision boundaries
- Which one is better?



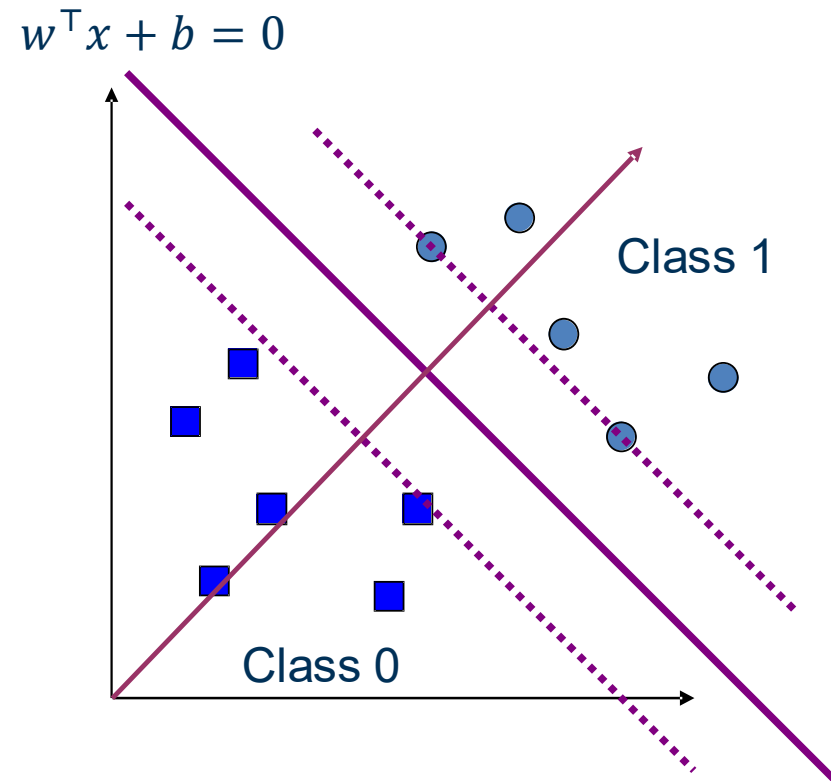
Which Decision Boundary is Better?

- Suppose we perturb the data, which boundary is more robust to error?



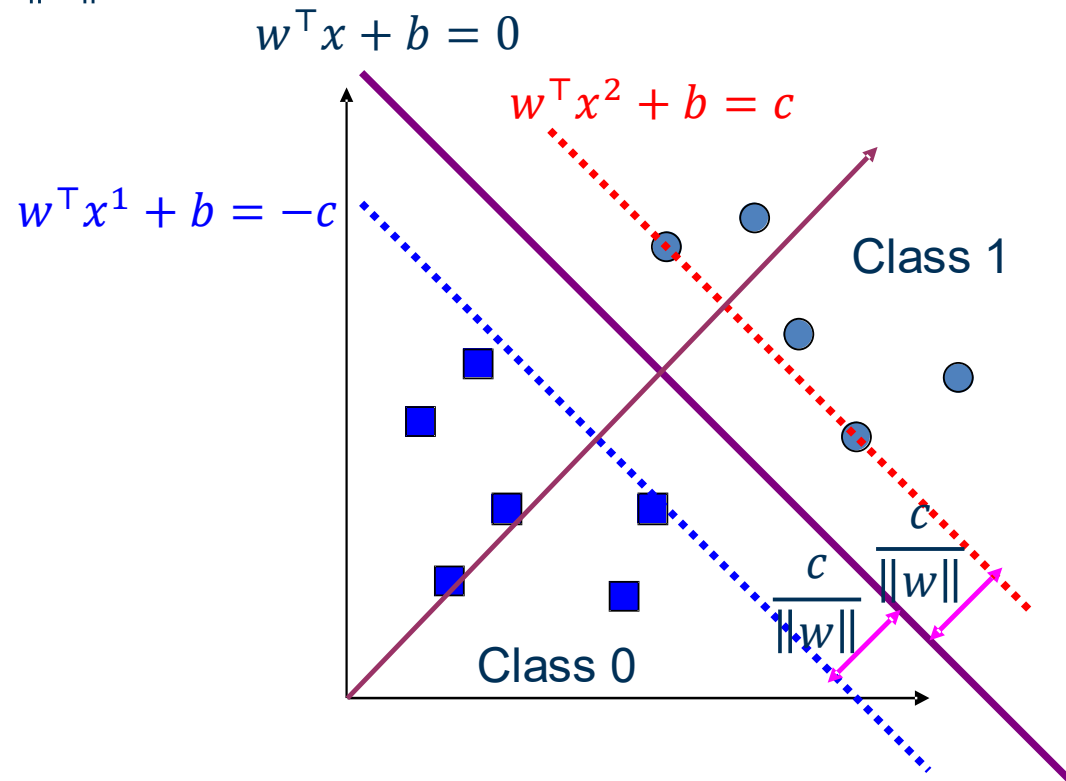
Geometric Interpretation of a Classifier

- Parameterizing decision boundary as: $w^T x + b = 0$
 - w denotes a vector orthogonal to the decision boundary
 - b is a scalar offset term
- Dash lines are parallel to the decision boundary, and they just hit the data points



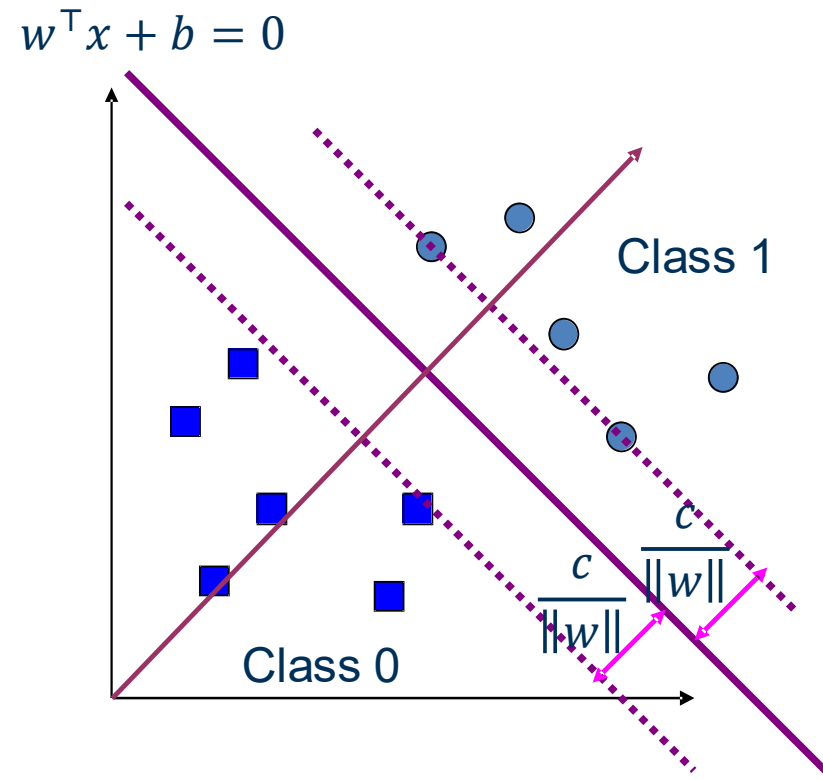
Formal Definition of Margin

- Pick two data points x^1 and x^2 which are on each dash line respectively
 - $w^\top x^2 + b = c$
 - $w^\top x^1 + b = -c$
- The unnormalized margin is $\tilde{\gamma} = w^\top (x^2 - x^1) = 2c$
- The margin is $\gamma = \frac{2c}{\|w\|}$



Formal Definition of Constraints on Data Points

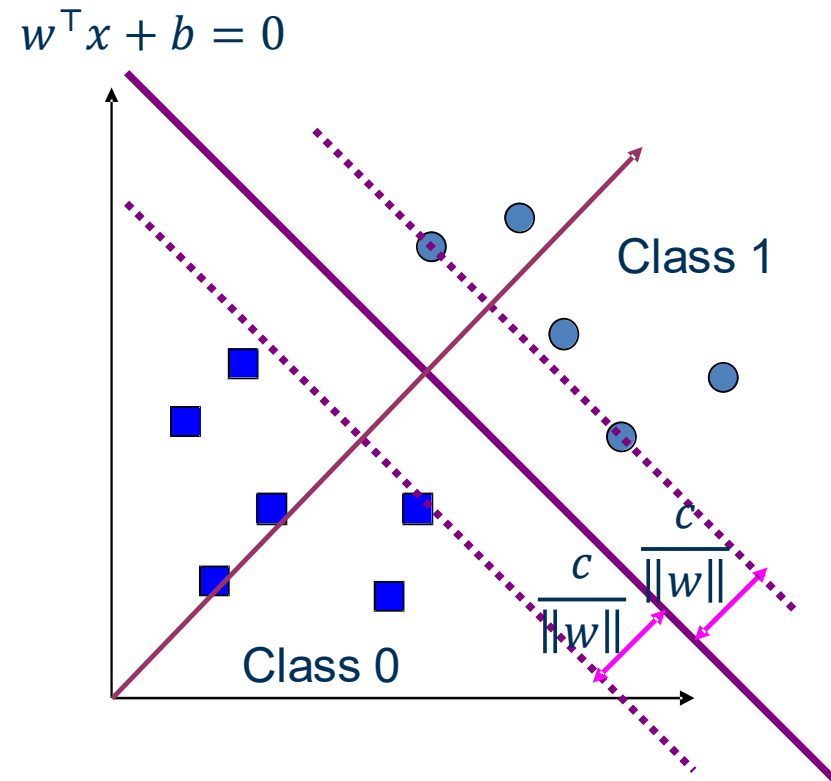
- Constraints on data points
 - For all x in class 1, $y = 1$ and $w^\top x + b \geq c$
 - For all x in class 0, $y = -1$ and $w^\top x + b \leq -c$
- Or more compactly, $(w^\top x + b)y \geq c$



Maximum Margin Classifier

- Find decision boundary w as far from data point as possible

$$\begin{aligned} \max_{w,b} \quad & \gamma = \frac{2c}{\|w\|} \\ \text{s.t.} \quad & y^i(w^\top x^i + b) \geq c, \quad \forall i \end{aligned}$$



Equivalent Form

$$\max_{w,b} \frac{2c}{\|w\|}$$
$$s.t. y^i(w^\top x^i + b) \geq c, \quad \forall i$$

- Note that the magnitude of c merely scales w and b , and does not change the relative goodness of different classifiers
- Set $c = 1$ and drop the constant to get a cleaner problem

$$\max_{w,b} \frac{1}{\|w\|}$$
$$s.t. y^i(w^\top x^i + b) \geq 1, \quad \forall i$$

Support Vector Machine Optimization Form

- A constrained convex quadratic programming problem (standard form)

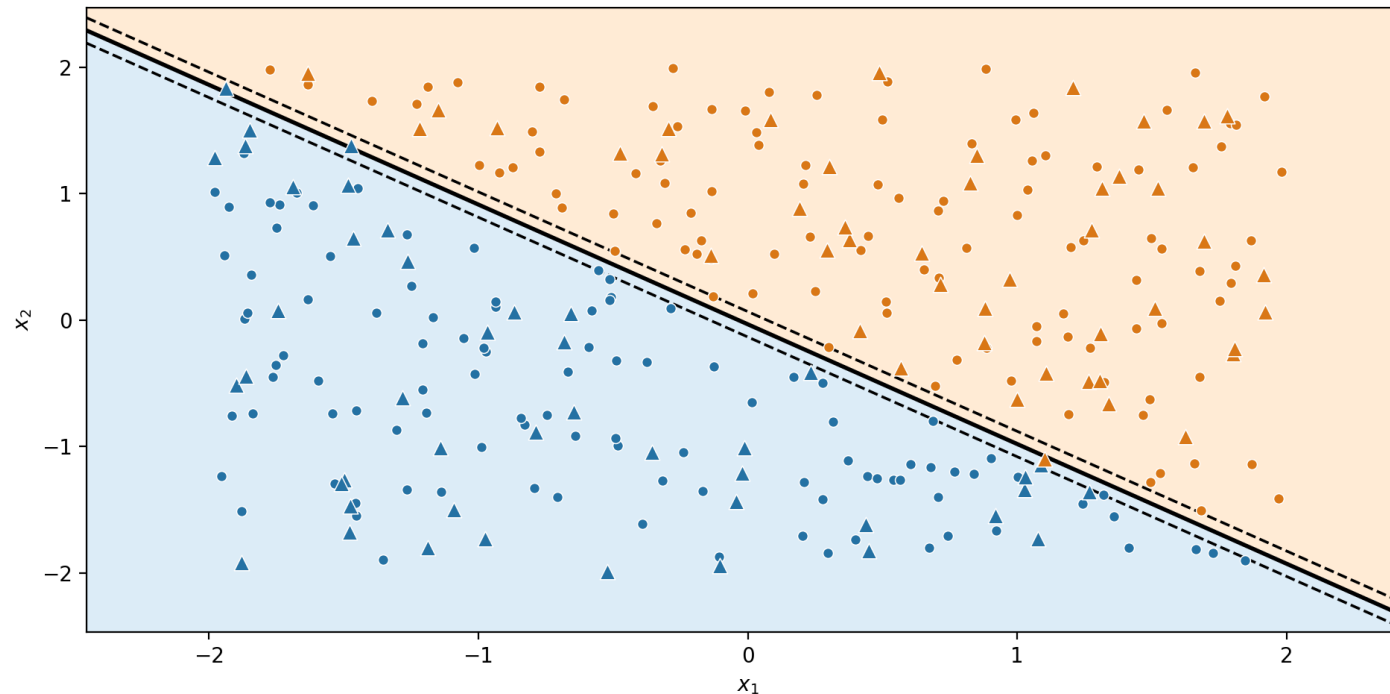
$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & 1 - y^i (w^\top x^i + b) \leq 0, \quad \forall i \end{aligned}$$

- After optimization, the margin is given by $\frac{2}{\|w\|}$

Support Vector Machine

Synthetic: diagonal separation

Hard-margin SVM | Optimizer iteration 96 / 96
Solid: $f = 0$; dashed: $f = \pm 1$; colors: predicted class



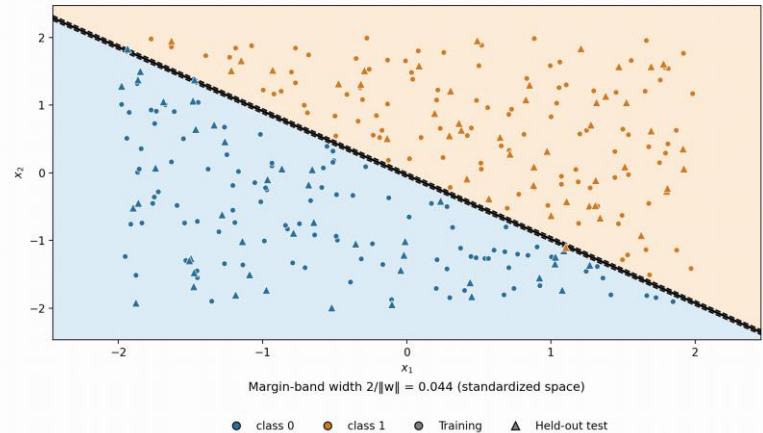
Margin-band width $2/\|w\| = 0.131$ (standardized space)

● class 0 ● class 1 ● Training ▲ Held-out test

Support Vector Machine

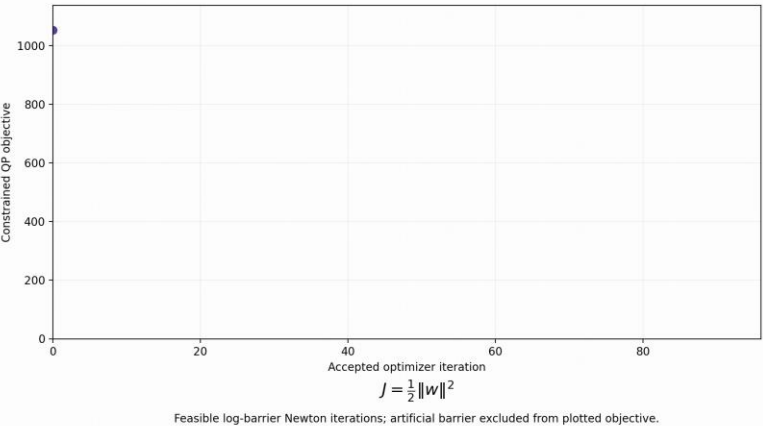
Synthetic: diagonal separation

Hard-margin SVM | Optimizer iteration 0 / 96
Solid: $f = 0$; dashed: $f = \pm 1$; colors: predicted class



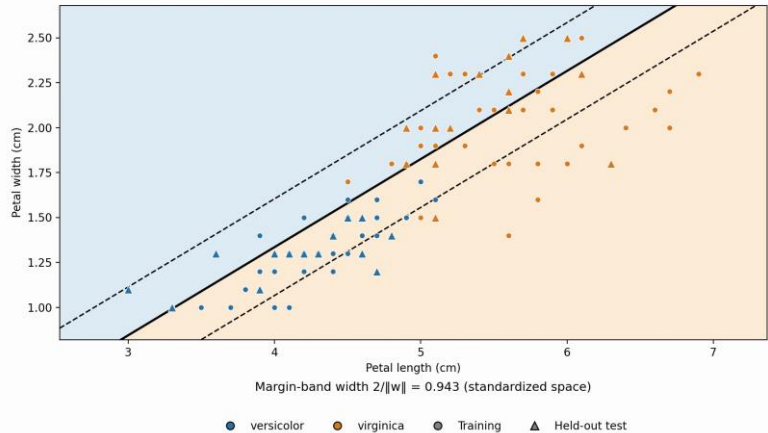
Synthetic: diagonal separation

Hard-margin SVM | Optimizer iteration 0 / 96
Objective = 1054.0104



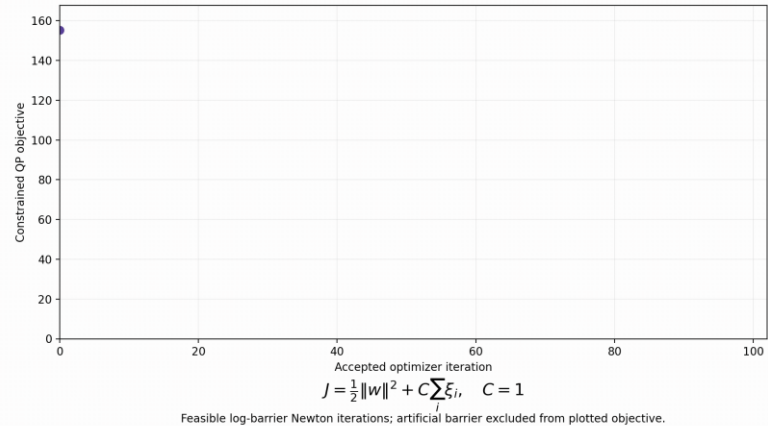
Iris: two species, two measurements

Soft-margin SVM ($C = 1$) | Optimizer iteration 0 / 102
Solid: $f = 0$; dashed: $f = \pm 1$; colors: predicted class



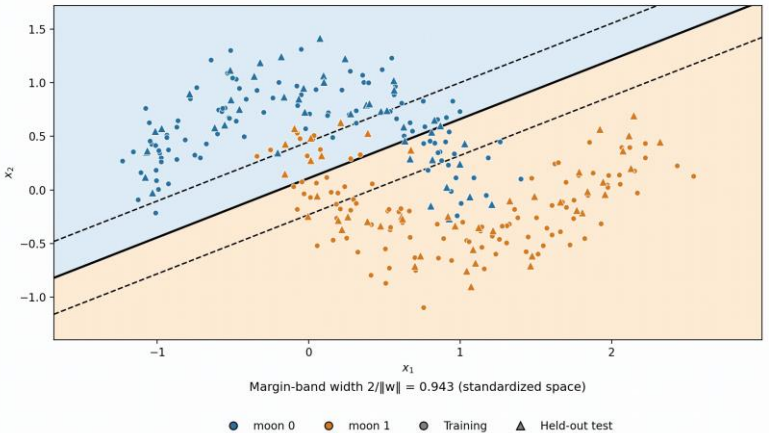
Iris: two species, two measurements

Soft-margin SVM ($C = 1$) | Optimizer iteration 0 / 102
Objective = 155.3358



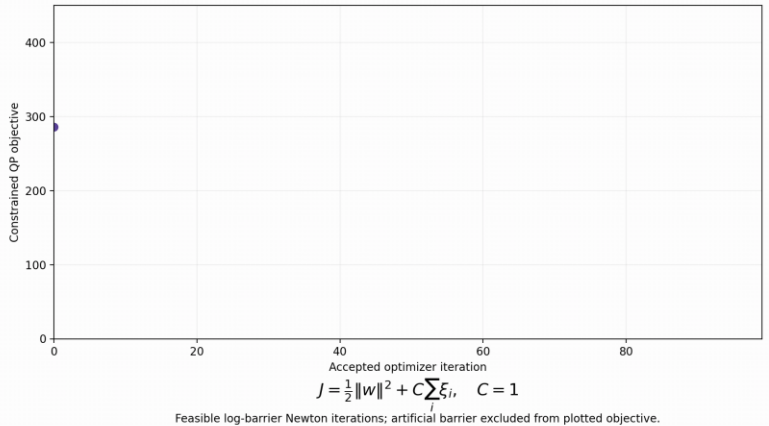
Synthetic: two interleaving moons

Soft-margin SVM ($C = 1$) | Optimizer iteration 0 / 99
Solid: $f = 0$; dashed: $f = \pm 1$; colors: predicted class



Synthetic: two interleaving moons

Soft-margin SVM ($C = 1$) | Optimizer iteration 0 / 99
Objective = 285.8587



What if the points are not fully separable?

